Alternating Least Squares for Collaborative Filtering: Research and Design Report

Algorithm design only. No code and no experimental results are included.

Section A: Research Questions

1. What is ALS and what problem does it solve?

ALS stands for Alternating Least Squares. It is a method used in collaborative filtering, which means recommending items to a user based on the ratings of many users. The data is a big table with users as rows and items as columns. Most of the table is empty because each user only rates a few items. ALS fills in the empty spots by guessing what rating a user would give to an item they have not seen (Koren, Bell & Volinsky, 2009).

2. What is matrix factorization?

Matrix factorization splits the big rating table R into two smaller tables, one for users and one for items. The user table P has one row per user and k columns, so its shape is m × k. The item table Q has one row per item and k columns, so its shape is n × k. The number k is the latent-factor dimension, which is the number of hidden features we learn for each user and item.

A row of P is the **user factors** for one user. A row of Q is the **item factors** for one item. The hidden features are not named by us. For movies they might loosely mean things like "how much action" or "how serious." The predicted rating is found by multiplying the two rows together:

R ≈ P × Q^T, with shapes (m × k) × (k × n) = (m × n)

rating guess for user u and item i = (user u's factors) · (item i's factors)

3. Why is it called "alternating"?

We need to find both P and Q, but finding both at the same time is hard. If we keep Q fixed, finding P becomes an ordinary least-squares problem, which has a direct answer. The same is true for Q when P is fixed. So ALS takes turns: fix Q and solve for P, then fix P

and solve for Q, and repeat. Each turn can only keep the error the same or make it smaller (Koren et al., 2009). Also, with Q fixed, every user can be solved on their own, so the work can run in parallel (Zhou et al., 2008).

4. Loss function, regularization and missing ratings

The loss function measures how wrong the model is. For explicit ratings it is the squared difference between the real rating and the predicted rating, added up over the ratings we actually have. Regularization adds a penalty for large factor values:

loss = sum of (real rating − predicted rating)² over observed ratings + λ × (sum of squared factor values)

The value λ controls how strong the penalty is. Without regularization the model can memorize the training ratings and then do badly on new ones (overfitting).

Missing entries should not be treated as zero ratings. An empty spot does not mean the user disliked the item. It usually means the user never saw or never rated it. If we filled blanks with zero, the model would learn that unseen items are bad and its predictions would be too low. So the loss only uses the observed ratings.

5. When is ALS useful? Comparison with neighborhood methods

ALS works well when there is a lot of data that is mostly empty, and when we want fast predictions after training. Here is a simple comparison with neighborhood collaborative filtering (finding similar users or items).

	ALS	Neighborhood method
How it works	Learns hidden features for users and items	Finds similar users or items
Sparse data	Handles it fairly well	Weak, because few shared ratings make similarity unreliable
Large data	Scales well, can run in parallel	Similarity calculations get slow
Explaining results	Hard, features are abstract	Easy ("people like you liked this")
Setup	Needs tuning and training time	Simple, few settings

The main limits of ALS are the tuning work, the lack of easy explanations, and that it cannot handle users or items with no ratings.

6. Explicit vs. implicit feedback

Explicit feedback is when users clearly tell us what they think, such as a 1 to 5 star rating. **Implicit feedback** is when we only see behavior, such as clicks or views. Implicit data is easy to collect but unclear: a click does not always mean the user liked the item, and not clicking does not always mean dislike (Hu, Koren & Volinsky, 2008).

Because of this, implicit ALS is set up differently:

1. Each interaction becomes a simple preference: 1 if the user interacted, 0 if not.
2. Each pair gets a confidence value that grows with how often the user interacted.
3. The loss uses all user-item pairs, not only observed ones, so the zeros count as weak evidence of no interest.

7. Hyperparameters

- **Rank k.** If k is too small, the model is too simple and misses patterns (underfitting). If k is too large, it overfits and is slower.
- **Regularization λ.** If λ is too small, the model overfits. If λ is too large, the factors shrink too much and all predictions end up near the average.
- **Number of iterations.** Too few and the model has not finished learning. Too many wastes time, and the error usually stops improving after a while anyway.

For implicit ALS there is also the confidence scale α, which controls how much we trust the interactions.

8. Cold start, popularity and sparsity

- **Cold start:** a new user or item has no ratings, so ALS has nothing to learn from and cannot make a good prediction.
- **Popularity imbalance:** a few items get most of the ratings. Their factors are learned well, while rarely rated items are learned poorly, so popular items get recommended too often.
- **Sparsity:** if users have only a few ratings each, the factors are poorly determined, so the model overfits unless k is small or λ is large.

Fallback for brand-new users: at first, recommend the most popular or highest-rated items (using an average that is adjusted so items with very few ratings do not rank too high). Then ask the new user to rate a few items. Once they have a few ratings, we can calculate their factors using the fixed item factors, without retraining everything.

Section B: Implementation Plan

This plan is for explicit-feedback ALS on a rating dataset made of (user, item, rating) records, for example MovieLens.

Step 1. Load and clean the data

- **Purpose:** get a clean list of observed ratings.
- **Input:** the raw rating file.
- **Output:** a table of user, item, rating, and time with bad rows removed.
- **Check:** ratings are in the valid range (1 to 5), no missing IDs, and duplicates are removed. Count the users, items and ratings.

Step 2. Map IDs and separate observed from missing values

- **Purpose:** turn user and item IDs into simple numbers 0, 1, 2, and so on.
- **Input:** the cleaned table.
- **Output:** lookup tables for users and items, and a list of each user′s ratings and each item′s ratings.
- **Check:** each ID has exactly one number. Only observed ratings are stored, and missing ones are not turned into zeros.

Step 3. Split into training, validation and test sets

- **Purpose:** test the model on ratings it has never seen.
- **Input:** the mapped ratings.
- **Output:** three separate sets, for example 80% / 10% / 10%, split per user or by time.
- **Check:** no rating appears in two sets. Anything calculated for training (such as the average rating) uses the training set only, to avoid leakage. Users or items that appear only in the test set are noted as cold-start cases.

Step 4. Decide how to evaluate

- **Purpose:** set the rules for measuring results before tuning.
- **Input:** the validation and test sets.
- **Output:** RMSE and MAE as the measures, plus simple baselines such as the global average rating.
- **Check:** tune only on the validation set and use the test set once at the end. Keep predictions within the rating range.

Step 5. Choose the factor sizes and starting values

- **Purpose:** create P (m × k) and Q (n × k).
- **Input:** the number of users m, the number of items n, and a value for k.
- **Output:** P and Q filled with small random numbers.
- **Check:** the values are not all zero or all the same, and the random seed is written down so results can be repeated.

Step 6. Choose the settings to try

- **Purpose:** limit how much tuning is needed.
- **Input:** the size of the dataset.
- **Output:** a short list of values, for example k = 10, 20, 50 and λ = 0.01, 0.1, 1, with a maximum number of iterations.
- **Check:** the list includes values that are too small and too large, so both underfitting and overfitting can be seen.

Step 7. Update the user factors

- **Purpose:** improve P while Q is fixed.
- **Input:** the current Q, each user's observed ratings, and λ.
- **Output:** a new P, found by solving a small k × k least-squares problem for each user using only the items that user rated.
- **Check:** only observed ratings are used. Users with no training ratings are skipped and noted.

Step 8. Update the item factors

- **Purpose:** improve Q while P is fixed.

- **Input:** the new P, each item′s observed ratings, and λ.
- **Output:** a new Q, found the same way as Step 7 but for each item.
- **Check:** Steps 7 and 8 together count as one iteration, and the training loss should not go up after an iteration. If it does, something is wrong.

Step 9. Monitor the error, stop and tune

- **Purpose:** decide when to stop and which settings are best.
- **Input:** training and validation RMSE after each iteration.
- **Output:** a record of the error for each setting and the best choice of k and λ.
- **Check:** stop when validation RMSE no longer improves much for a few iterations, or when the maximum is reached. If training error drops but validation error rises, raise λ or lower k. Pick the setting with the lowest validation error.

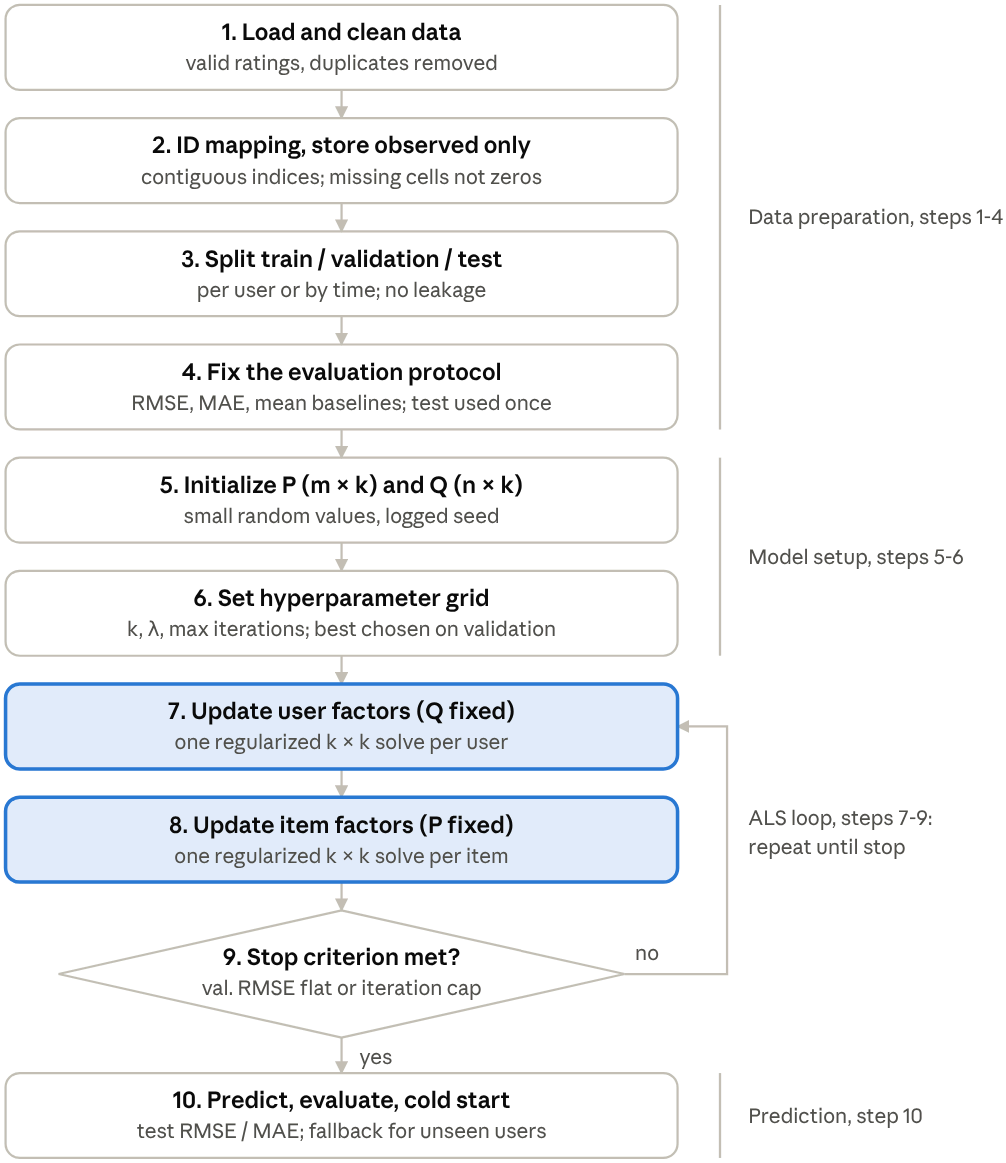
Step 10. Predict, evaluate and handle cold start

- **Purpose:** make final predictions and report how good they are.
- **Input:** the final P and Q, and the test set.
- **Output:** the predicted rating for user u and item i is the dot product of u′s factors and i′s factors, kept within the rating range. Then calculate RMSE and MAE on the test set and compare them to the baselines. For a new user or item, use the fallback from Section A (popular items or the average rating).
- **Check:** report how many test cases used the fallback, and confirm the model beats the baselines before calling it good. Do not change settings based on the test set.

Figure 1. Algorithm flowchart

(See the flowchart below.)

ALS training repeats two solves until validation error stops improving



ALS algorithm flowchart · 10 steps, 1 loop

Steps 1 to 6 prepare the data and set up the model; steps 7 to 9 (highlighted) alternate user and item solves until the stop criterion is met; step 10 produces predictions and the final evaluation.

References

Apache Spark. (n.d.). *Collaborative filtering*. In *MLlib: Main Guide*. Apache Software Foundation. https://spark.apache.org/docs/latest/ml-collaborative-filtering.html

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008)* (pp. 263-272). IEEE. https://doi.org/10.1109/ICDM.2008.22

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30-37. https://doi.org/10.1109/MC.2009.263

Zhou, Y., Wilkinson, D., Schreiber, R., & Pan, R. (2008). Large-scale parallel collaborative filtering for the Netflix Prize. In *Algorithmic Aspects in Information and Management (AAIM 2008)*, LNCS 5034 (pp. 337-348). Springer. https://doi.org/10.1007/978-3-540-68880-8_32